

Letter to the Team Kilo

Use Case Descriptions & Diagram

General Critique

The use case descriptions for leaderboard and matchmaking effectively outline the core functionalities of ranking players and pairing them for competitive play. However, certain design choices may impact system efficiency and user experience. The leaderboard system primarily tracks player performance, but filtering and rank history features could be better integrated to enhance clarity and usability.

Feature Requests

- Allow users to compare their stats with another player's (e.g., win rate, ELO, total matches played) directly from the leaderboard or profile page. This would add a competitive aspect and provide more context to rankings.
- Implement a matchmaking history feature allowing players to track previous opponents and match outcomes.

Constructive Feedback

Some use case triggers lack clarity, with descriptions resembling functionality explanations rather than defining when a process starts. Providing explicit conditions for entering and exiting queues would improve readability. Additionally, matchmaking edge cases (e.g., handling players who leave mid-queue or refuse matches) should be more explicitly defined instead of left as open issues.

Class Structure Diagram

General Critique

Strengths:

The matchmaking system appears to have a clear class hierarchy, with major components like Player, Match, and Leaderboard explicitly stated. The relationships between different aspects (such as Matchmaking, Leaderboard, and Player) appear to be well-connected, indicating a functional matchmaking system.

Weaknesses:

The Leaderboard class should have crucial features such as a ranking system, score tracking, and update procedures. There is no apparent system for recording past match results, which is necessary for tracking player performance over time. If this system is intended for a large-scale multiplayer game, efficiency in matchmaking should be addressed through load balancing and queue management.

Feature Requests

- Store previous match results so that players can see their win/loss record and performance patterns, which will help with ranking computations and matching modifications.
- Implement a queuing mechanism that prioritizes long-waiting players while gradually expanding search criteria to balance fairness and shorten wait times.

Constructive Feedback

Improve code modularity by dividing matchmaking logic into separate components such as QueueManager and Matchmaker. Use design patterns such as Strategy to create customizable matchmaking algorithms and Observer to get real-time player notifications. Optimize database queries for leaderboard updates to ensure peak performance, particularly in large-scale multiplayer scenarios.